Part C: Object-Oriented Programming (OOP) Master Interview Q&A Guide (Q1 – Q126)

Comprehensive Compilation covering Encapsulation, Constructors, Inheritance, Abstraction, Polymorphism, Method Overloading & Overriding | Notes Pages 105–140

1. Encapsulation

———————————————————————————————————————

Q1. What is encapsulation in Java?

Ans: Encapsulation data (variables) aur uspar kaam karne wale code (methods) ko ek single unit yaani class ke andar bundle karta hai aur data ko directly outside access hone se rokta hai. Variables ko private declare kiya jaata hai aur public methods ke zariye controlled access diya jaata hai.

Q2. How do you implement encapsulation in Java?

Ans: Class ke fields ko 'private' declare karein, aur unhe access/modify karne ke liye 'public' getter, setter ya specific business methods provide karein.

```
class BankAccount {
    private double balance;
    private String pin;

    public BankAccount(double balance, String pin) {
        this.balance = balance;
        this.pin = pin;
    }
    public double getBalance() { return balance; }
    public void deposit(double amount, String enteredPin) {
        if (pin.equals(enteredPin) && amount > 0) balance += amount;
    }
    public void withdraw(double amount, String enteredPin) {
        if (pin.equals(enteredPin) && amount <= balance && amount > 0) balance -=
amount;
    }
}
```

Q3. What is data hiding?

Ans: Data hiding ka matlab hai variables ko 'private' banakar outside classes se inaccessible rakhna taaki koi unhe directly modify ya misuse na kar sake.

Q4. Are encapsulation and data hiding exactly the same?

Ans: Nahi. Data hiding ek technique hai jisme direct access restrict kiya jaata hai. Encapsulation ek broader architectural concept hai jisme data hiding ke saath data aur behavior ko ek sath wrap kiya jaata hai.

Q5. Why should fields usually be private?

Ans: Private fields outside classes ko arbitrary ya corrupt state set karne se rokte hain, jisse class apne state ko validation rules ke mutabiq khud control karti hai.

Q6. What is a getter?

Ans: Getter ek public method hai jo private variable ki value return karta hai (e.g., public double getBalance() { return balance; }).

Q7. What is a setter?

Ans: Setter ek public method hai jo private variable ki value update karta hai, aksar validation logic ke sath (e.g., public void setName(String name) { this.name = name; }).

Q8. Do all private fields need both getters and setters?

Ans: Nahi. Sirf wahi access provide karna chahiye jiski zaroorat ho. Read-only ke liye sirf getter aur write-only ke liye sirf setter diya jaata hai.

Q9. How does encapsulation improve security?

Ans: Yeh outside code ko directly sensitive data manipulate karne se rokta hai aur har modification se pehle strict authentication ya validation check enforce karta hai.

Q10. How does encapsulation improve maintainability?

Ans: Internal implementation details ko bina kisi caller code ko break kiye freely modify kiya ja sakta hai, jab tak public method signatures same rehte hain.

Q11. Give a real-world example of encapsulation from the notes.

Ans: ATM ya Bank Account: Balance aur PIN private hote hain. User directly balance modify nahi kar sakta, sirf deposit() aur withdraw() methods ke zariye check aur balance updates kar sakta hai.

Q12. How would you design an encapsulated BankAccount?

Ans: Balance ko private rakhein, getBalance() expose karein, aur deposit() / withdraw() methods mein positive amount check aur balance sufficiency validation add karein.

Q13. Can another class directly access a private field?

Ans: Nahi, private members sirf usi declaring class ke scope ke andar accessible hote hain.

Q14. Can a setter contain validation logic?

Ans: Haan, setter ke andar age checking, non-negative checks, ya security conditions validate ki ja sakti hain.

Q15. Does using getters and setters automatically make a class well encapsulated?

Ans: Nahi. Agar setters bina validation ke kisi bhi invalid value ko accept kar lete hain, to encapsulation effectively weak ho jaata hai.

Q16. What is the difference between a public field and a private field with a getter/setter?

Ans: Public field uncontrolled direct modification allow karta hai, jabki private field with methods controlled access, logging, aur validation enforce karta hai.

Q17. Can a getter modify an object's state?

Ans: Technically kar sakta hai, lekin standard conventions ke mutabiq getter ko hamesha side-effect free hona chahiye aur sirf state read karni chahiye.

Q18. Why might a PIN not have a public getter?

Ans: PIN sensitive credential hai. Uska raw getter expose karna security violate karta hai; verification internally boolean match se honi chahiye.

Q19. What interview example demonstrates controlled access?

Ans: ATM withdrawal process: entered PIN check karna, positive amount verify karna, balance sufficiency check karna, aur tab internal balance deduct karna.

Q20. What is the main benefit of encapsulation in one sentence?

Ans: Yeh data aur behaviour ko bundle karke class ke internal state ko unauthorized modification se protect karta hai.

Q21. What is the difference between encapsulation and abstraction?

Ans: Encapsulation implementation level par data hiding karti hai (private + getters/setters), jabki Abstraction design level par system ki internal complexity ko hide karti hai (abstract classes & interfaces).

Q22. How do you make a class immutable?

Ans: Class ko final mark karein, saare fields private final karein, koi setter na dein, initialization constructor se karein, aur mutable objects ke defensive copies return karein.

Q23. Can you have encapsulation without getters and setters?

Ans: Haan, encapsulation ke liye private data aur controlled behavior zaroori hai, jo business methods (jaise deposit(), withdraw()) se easily ho sakta hai.

Q24. What is a fully encapsulated class?

Ans: Wo class jisme saare instance variables private declare kiye gaye hon.

Q25. Which access modifiers are used for encapsulation?

Ans: Primary data hiding ke liye 'private', access methods ke liye 'public', aur protected/package-private controlled inheritance scenarios ke liye.

2. Constructors

———————————————————————————————————————

Q26. What is a constructor in Java?

Ans: Constructor ek special member block hai jo object create hote waqt invoke hota hai aur object ke instance fields ko initialize karta hai. Iska naam class ke naam jaisa hota hai aur koi return type nahi hota.

Q27. When is a constructor called?

Ans: Jab bhi 'new' keyword se object banaya jaata hai, heap par memory allocate hote hi constructor automatically call hota hai.

Q28. How is a constructor different from a method?

Ans: Constructor ka naam class ke naam jaisa hota hai, return type nahi hota, 'new' se auto-invoke hota hai, aur inherit nahi hota. Method ka apna custom naam aur return type hota hai aur explicitly call hota hai.

Q29. What is a no-argument constructor?

Ans: Wo constructor jo koi parameters accept nahi karta. Yeh developer explicitly code kar sakta hai ya compiler implicitly provide karta hai.

Q30. What is the difference between a default constructor and a no-argument constructor?

Ans: Default constructor compiler tab automatically provide karta hai jab class mein koi bhi constructor na likha ho. No-arg constructor developer dwara explicitly likha gaya bina parameter wala constructor hota hai.

Q31. When does the compiler provide a default constructor?

Ans: Sirf tab jab class ke andar developer ne koi bhi constructor define na kiya ho.

Q32. What is a parameterized constructor?

Ans: Wo constructor jo arguments accept karta hai taaki object creation ke waqt custom values se fields ko initialize kiya ja sake.

```
class Account {
    int accNo;
    String name;
    Account() { } // no-arg
    Account(int accNo, String name) { // parameterized
        this.accNo = accNo;
        this.name = name;
    }
}
```

Q33. Why do we need constructors?

Ans: Bina constructor ke objects default values (0, null) ke sath bante hain. Constructors ensure karte hain ki object initialization ke sath hi ek valid usable state mein ho.

Q34. Can a constructor have a return type?

Ans: Nahi, yahan tak ki 'void' bhi nahi. Return type likhte hi compiler use standard method treat karne lagta hai.

Q35. Can a constructor be static, final, abstract, or synchronized?

Ans: Nahi, constructors in charon modifiers ke sath declare nahi kiye ja sakte.

Q36. Can a constructor be private?

Ans: Haan. Private constructor outside instantiation ko block karta hai, jo Singleton pattern aur utility classes mein use hota hai.

Q37. Can a constructor throw an exception?

Ans: Haan, constructor checked ya unchecked exceptions throws clause ke zariye declare aur throw kar sakta hai.

Q38. Can we call a constructor explicitly using an existing object?

Ans: Nahi. Constructor sirf 'new' keyword, 'this()', ya 'super()' ke zariye call ho sakta hai; 'obj.Account()' likhna compile error hai.

Q39. What is constructor chaining?

Ans: Ek constructor se doosre constructor ko invoke karne ki technique ko constructor chaining kehte hain.

Q40. What does this() do in a constructor?

Ans: 'this()' usi class ke kisi doosre constructor ko call karta hai aur yeh strictly constructor ka pehla statement hona chahiye.

Q41. What does super() do in a constructor?

Ans: 'super()' immediate superclass ke constructor ko call karta hai aur yeh bhi constructor ka first statement hona chahiye.

Q42. Can this() and super() both be written together in one constructor?

Ans: Nahi, kyunki dono ko hi first statement hona compulsory hai, isliye ek constructor mein dono sath nahi aa sakte.

Q43. What happens if a subclass constructor does not explicitly call super()?

Ans: Compiler automatically pehli line par no-argument 'super()' insert kar deta hai. Agar parent class mein no-arg constructor nahi hai, to compile-time error aati hai.

Q44. What is the execution order in constructor chaining?

Ans: Execution top-down chalti hai: Parent class constructor pehle execute hota hai, uske baad child class constructor body run hoti hai.

Q45. Can a constructor be overloaded?

Ans: Haan, ek hi class mein different parameter count, types, ya order ke sath multiple constructors define kiye ja sakte hain.

Q46. Can constructors be overridden?

Ans: Nahi, constructors child class mein inherit nahi hote, isliye override bhi nahi ho sakte.

Q47. What happens if I define only Account(String name) and then write new Account()?

Ans: Compile-time error aati hai, kyunki parameterized constructor likhte hi compiler default constructor generate karna band kar deta hai.

Q48. What is the role of constructors in a Singleton pattern?

Ans: Constructor ko private banaya jaata hai taaki bahar se koi 'new' keyword use na kar sake, aur instance creation class ke static method (e.g., getInstance()) se control ho sake.

```
class Singleton {
    private static Singleton instance;
    private Singleton() { }
    public static Singleton getInstance() {
        if (instance == null) instance = new Singleton();
        return instance;
    }
}
```

Q49. What is recursive constructor invocation?

Ans: Jab constructor A 'this()' se constructor B ko call kare aur B wapas A ko call kare, to compiler 'recursive constructor invocation' error throw karta hai.

Q50. What is a copy constructor? Does Java provide one?

Ans: Wo constructor jo usi class ke dusre object ko parameter ke roop mein lekar fields copy karta hai. Java ise automatically provide nahi karta, manually likhna padta hai.

Q51. Can an abstract class have a constructor?

Ans: Haan, abstract class ka constructor hota hai jo concrete subclass ke instance bante waqt super() ke through common fields initialize karta hai.

Q52. Can an interface have a constructor?

Ans: Nahi, interfaces ke paas koi constructor nahi hota.

3. Inheritance

———————————————————————————————————————

Q53. What is inheritance in Java, and why do we need it?

Ans: Inheritance ek OOP mechanism hai jisme child class parent class ke accessible properties aur methods acquire karti hai ('extends' keyword se). Yeh code reusability, runtime polymorphism aur clean domain modeling ke liye zaroori hai.

Q54. Which keyword is used for inheritance? Give syntax.

Ans: 'extends' keyword use hota hai:
class Parent { }
class Child extends Parent { }

Q55. What are the terms used in inheritance?

Ans: Superclass / Parent / Base class (jisse inherit kiya ja raha hai) aur Subclass / Child / Derived class (jo inherit kar rahi hai).

Q56. What are the types of inheritance in Java?

Ans: Single, Multilevel, Hierarchical, Multiple (classes ke sath nahi, interface se), aur Hybrid (interface se).

Q57. What can a child class do when using inheritance?

Ans: Inherited accessible fields/methods ko directly use kar sakti hai, naye fields/methods add kar sakti hai, instance methods override kar sakti hai, aur static methods hide kar sakti hai.

Q58. Why is multiple inheritance not supported with classes in Java? (Diamond Problem)

Ans: Ambiguity avoid karne ke liye. Agar do parent classes mein identical method signature ho aur child dono ko extend kare, to child object par method call karne par compiler decide nahi kar pata ki kiska code run kare.

Q59. How does Java 8 handle the diamond problem with default methods?

Ans: Agar do implemented interfaces mein identical signature ka default method ho, to child class ko method override karke explicitly 'InterfaceName.super.methodName()' se ambiguity resolve karni padti hai.

```
interface A { default void show() { System.out.println("A"); } }
interface B { default void show() { System.out.println("B"); } }
class C implements A, B {
    @Override
    public void show() { A.super.show(); }
}
```

Q60. Can a subclass access private members of the parent?

Ans: Nahi. Subclass private members ko directly access nahi kar sakti, sirf parent ke accessible getters, setters ya methods ke through interact kar sakti hai.

Q61. What happens if parent and child have the same variable name?

Ans: Child variable parent variable ko hide kar deta hai (Variable Hiding). Variables polymorphic nahi hote; access reference type par depend karta hai. Inside child, this.x child ka aur super.x parent ka variable access karta hai.

Q62. What is the difference between method overriding and method hiding?

Ans: Overriding instance methods par hoti hai aur runtime par actual object ke hisaab se dynamically resolve hoti hai. Method hiding static methods par hoti hai aur compile-time par reference type se resolve hoti hai.

Q63. Can we override a private method?

Ans: Nahi. Private methods subclass ko visible nahi hote, isliye override nahi ho sakte. Subclass mein same naam se likha gaya method ek naya independent method mana jaata hai.

Q64. Can we override a static method?

Ans: Nahi. Static methods class level par hote hain, object level par nahi. Redefine karne par wo method hiding kehlaata hai.

Q65. What is the root class of all Java classes?

Ans: java.lang.Object class Java ki root class hai. Har class implicitly Object class ko extend karti hai.

Q66. Can a final class be extended or a final method overridden?

Ans: Nahi. 'final' class ko subclass nahi kiya ja sakta, aur 'final' method ko child class override nahi kar sakti.

Q67. What is the difference between IS-A and HAS-A relationships?

Ans: IS-A inheritance hai ('extends' / 'implements', jaise SavingsAccount IS-A Account). HAS-A composition hai jisme ek class dusri class ka reference hold karti hai (jaise Car HAS-A Engine).

Q68. Which members does a child NOT inherit?

Ans: Private members, constructors, aur static/instance initializer blocks child class inherit nahi karti.

Q69. What are the three uses of the super keyword?

Ans: 1. super.field se parent variable access karna.
2. super.method() se parent overridden method call karna.
3. super(...) se parent constructor call karna.

Q70. What is upcasting and downcasting?

Ans: Child object ko parent reference mein hold karna Upcasting hai (implicit & safe). Parent reference ko child type mein cast karna Downcasting hai (explicit, ClassCastException throw kar sakta hai agar type match na ho).

Q71. Is cyclic inheritance allowed in Java?

Ans: Nahi, cyclic inheritance (class A extends B aur class B extends A) compile-time error deta hai.

4. Abstraction

——

Q72. What is abstraction in Java? How is it implemented?

Ans: Abstraction essential functionality ko display karne aur internal implementation details ko hide karne ka concept hai ('what to do' vs 'how to do'). Yeh Abstract Classes (partial abstraction) aur Interfaces (full abstraction) se achieve hota hai.

Q73. What is an abstract class?

Ans: Wo class jise 'abstract' keyword se declare kiya gaya ho, jiska direct object create nahi ho sakta, aur jisme abstract methods ke saath concrete methods bhi ho sakte hain.

Q74. What is an abstract method?

Ans: Wo method jise 'abstract' keyword se declare kiya gaya ho, jiska koi implementation body ({}) nahi hota, aur jo semicolon (;) par terminate hota hai.

Q75. Can we create an object of an abstract class?

Ans: Nahi, abstract classes incomplete hoti hain isliye 'new AbstractClass()' illegal hai. Par unka reference variable use karke concrete subclass object hold kiya ja sakta hai.

Q76. Can an abstract class have constructors?

Ans: Haan, abstract class ka constructor hota hai jo concrete subclass ke instance bante waqt super() ke through common state initialize karta hai.

Q77. What happens if an abstract class has no abstract methods?

Ans: Yeh completely valid hai. Class tab bhi directly instantiate nahi ho sakti, jisse direct object creation block karne ke liye use kiya ja sakta hai.

Q78. Can an abstract class be declared final?

Ans: Nahi, compile error aayega. Abstract class subclass hone ke liye banti hai aur final keyword subclassing ko prohibit karta hai.

Q79. Can an abstract method be declared static, final, or private?

Ans: Nahi. Abstract method ko subclass dwara override karna mandatory hai, jabki static, final, aur private methods override nahi kiye ja sakte.

Q80. What happens if a subclass does not implement all abstract methods of its parent?

Ans: Subclass compile nahi hogi jab tak ki wo subclass bhi khud ko 'abstract' declare na kare.

Q81. Can an abstract class have static methods?

Ans: Haan, static methods class level par hote hain aur unhe AbstractClass.methodName() se directly call kiya ja sakta hai.

Q82. Can an abstract class have final methods?

Ans: Haan, concrete final methods abstract class mein allow hain taaki subclasses common implementation use kar sakein par override na karein.

Q83. Can an abstract class have a main() method? Can it be executed?

Ans: Haan, main() ek static method hai, isliye JVM bina object create kiye use directly execute kar sakti hai.

Q84. Can an abstract class implement an interface without implementing its methods?

Ans: Haan, abstract class interface methods ko unimplemented chhod sakti hai; unhe pehli non-abstract concrete subclass ko implement karna padega.

Q85. Can a top-level abstract class be private or static?

Ans: Nahi, top-level class sirf public ya default ho sakti hai. Nested/inner abstract classes private ya static ho sakti hain.

Q86. What is the difference between partial abstraction and 100% abstraction?

Ans: Abstract class 0 se 100% partial abstraction provide karti hai (state aur concrete methods ke sath). Interface traditionally 100% contract abstraction provide karta hai.

Q87. When to choose Abstract Class vs Interface?

Ans: Jab closely related classes ko common state/code share karna ho to Abstract Class chunein. Jab unrelated classes ke beech behavior/contract define karna ho ya multiple inheritance chahiye ho to Interface chunein.

Q88. How to design a Payment system using abstraction?

Ans: Ek abstract class Payment banayein jisme abstract method pay(double amount) ho, aur UPIPayment, CreditCardPayment subclasses use extend karke apni specific logic implement karein.

```
abstract class Payment {
    abstract void pay(double amount);
}
class UPIPayment extends Payment {
    void pay(double a) { System.out.println("Paid via UPI: " + a); }
}
class CreditCardPayment extends Payment {
    void pay(double a) { System.out.println("Paid via Card: " + a); }
}
```

Q89. Can we instantiate an abstract class using an anonymous class?

Ans: Haan, lekin technical taur par wo abstract class ka instance nahi balki ek anonymous subclass ka instance hota hai.

5. Polymorphism

——

Q90. What is polymorphism in Java?

Ans: Polymorphism ('many forms') wo ability hai jisme ek hi method name ya reference variable context aur underlying runtime object ke according alag-alag behaviors represent karta hai.

Q91. What are the types of polymorphism in Java?

Ans: 1. Compile-time Polymorphism (Method Overloading / Static Binding).
2. Runtime Polymorphism (Method Overriding / Dynamic Binding).

Q92. How is compile-time polymorphism achieved?

Ans: Method Overloading ke zariye. Compiler method call ke waqt passed arguments ke type aur count ke basis par compile time par hi exact target method select kar leta hai.

Q93. How is runtime polymorphism achieved?

Ans: Method Overriding aur Upcasting ke zariye. Runtime par JVM heap par □□□□□ actual object ke hisaab se method implementation dynamically execute karti hai.

Q94. What is Dynamic Method Dispatch?

Ans: Dynamic Method Dispatch wo runtime mechanism hai jisme kisi overridden method call ko compile-time reference type ke bajaye actual runtime object ke basis par resolve kiya jaata hai.

Q95. Explain the two-step internal working of polymorphism.

Ans: 1. Compile-Time: Compiler reference type ke andar check karta hai ki declared method signature exist karta hai ya nahi.
2. Runtime: JVM heap par actual object ko inspect karti hai aur uske vtable se overridden method invoke karti hai.

Q96. What happens when a parent reference holds a child object?

Ans: Parent reference se sirf parent mein defined methods hi call ho sakte hain. Agar child ne unhe override kiya hai, to child ka version run hoga. Child-specific new methods bina downcasting ke access nahi ho sakte.

Q97. If Parent p = new Child(), which method runs?

Ans: Agar method child mein override kiya gaya hai, to Child class ka implementation execute hoga.

Q98. Is inheritance mandatory for runtime polymorphism?

Ans: Haan, kyunki dynamic dispatch ke liye superclass/interface reference aur subclass overridden method ka hona zaroori hai.

Q99. Are variables polymorphic in Java?

Ans: Nahi, variables polymorphic nahi hote. Instance variables compile-time par strictly reference type ke basis par resolve hote hain.

Q100. Compile-time vs Runtime polymorphism: which is faster?

Ans: Compile-time polymorphism faster hoti hai kyunki compiler pehle hi binding complete kar leta hai, jabki runtime polymorphism mein method table lookup ka slight overhead hota hai.

Q101. What is early binding vs late binding?

Ans: Early binding compile-time binding hai (overloaded, static, final, private methods). Late binding runtime dynamic binding hai (overridden instance methods).

Q102. What are the advantages and disadvantages of polymorphism?

Ans: Advantages: Code reusability, modularity, flexibility, extensibility.
Disadvantages: Tracing complex ho sakti hai aur runtime table lookup ka minor performance overhead hota hai.

6. Method Overloading

——

Q103. What is method overloading?

Ans: Ek hi class mein multiple methods ka same naam hona lekin unke parameters list (number, type, ya sequence) ka distinct hona.

Q104. What are the ways to overload a method?

Ans: 1. Parameters ka number change karke.
2. Parameters ka data type change karke.
3. Parameters ka sequence/order change karke.

Q105. Can method overloading be done by changing only the return type?

Ans: Nahi. Return type method signature ka part nahi hota. Agar sirf return type alag ho aur parameters same hon, to compiler duplicate method error throw karta hai.

Q106. Can we overload static methods in Java?

Ans: Haan, static methods ko parameter list change karke bilkul instance methods ki tarah overload kiya ja sakta hai.

Q107. Can we overload the main() method in Java?

Ans: Haan, different parameters ke sath multiple main() methods likhe ja sakte hain, lekin JVM execution hamesha standard 'public static void main(String[] args)' se hi start karti hai.

Q108. What is a method signature in Java?

Ans: Method signature method name aur parameter list (types and sequence) se banta hai. Return type aur access modifiers iska hissa nahi hote.

Q109. Can we overload a method by changing parameter names or modifiers (like final) only?

Ans: Nahi, parameter names ya parameter modifiers (jaise final) change karne se method signature change nahi hota.

Q110. How does Java resolve method matching when no exact match is found?

Ans: Java Type Promotion rules follow karti hai: 1. Widening primitive (int -> long -> float -> double) -> 2. Autoboxing/Unboxing (int -> Integer) -> 3. Varargs (int...).

Q111. What is an ambiguous method call error?

Ans: Jab do overloaded methods equally specific candidates ban jaate hain aur compiler decide nahi kar pata ki kisse call karein (e.g., test(String) aur test(Integer) ke sath test(null) call karna).

Q112. Is overloading resolved at compile time or runtime?

Ans: Strictly compile time par, reference type ke declared arguments ke basis par.

Q113. Can overloaded methods have different access modifiers or throw different exceptions?

Ans: Haan, parameter lists different hone par access modifiers aur exception specifications kuch bhi ho sakte hain.

Q114. Does overloading require inheritance?

Ans: Nahi, overloading single class ke scope mein hoti hai (haalanki subclass bhi parent method ko overload kar sakti hai).

7. Method Overriding

———————————————————————————————————————

Q115. What is method overriding in Java?

Ans: Jab subclass apne superclass ke kisi instance method ko same signature ke sath naya implementation provide karti hai taaki specialized behavior achieve ho sake.

Q116. What are the essential rules for method overriding?

Ans: 1. Signature identical hona chahiye.
2. Inheritance relation mandatory hai.
3. Access modifier parent se more restrictive nahi ho sakta.
4. Return type same ya covariant subtype hona chahiye.
5. Broader/new checked exception throw nahi kar sakta.
6. Static, private, final methods override nahi ho sakte.
7. Constructors override nahi ho sakte.

Q117. What is a Covariant Return Type?

Ans: Overriding method parent method ke declared return type ka subtype return kar sakti hai (reference types ke liye allow hai).

```
class A { Number get() { return 1; } }
class B extends A { @Override Integer get() { return 2; } } // Integer is subtype of
Number
```

Q118. Can an overriding method have a more restrictive access modifier?

Ans: Nahi. Visibility widen ho sakti hai (protected -> public), lekin restrict nahi ho sakti (public -> private compilation error deta hai).

Q119. What are the exception rules in method overriding?

Ans: Subclass method parent method se broader ya naye checked exceptions declare nahi kar sakti. Narrower checked exceptions ya koi bhi unchecked exceptions freely throw kar sakti hai.

Q120. Can static methods be overridden? (Concept of Method Hiding)

Ans: Nahi. Static methods class level par hote hain. Subclass mein identical static method likhna Method Hiding kehlata hai, jo compile-time reference type se bind hota hai.

Q121. What is the role of the @Override annotation?

Ans: Yeh compiler ko check karne bolta hai ki method successfully superclass method ko override kar raha hai ya nahi. Agar spelling ya parameter mein mistake ho, to turant compile error deta hai.

Q122. Can a final method or private method be overridden?

Ans: Nahi. Final method overriding ko strictly prohibit karta hai aur private method child class ko inherit/visible hi nahi hota.

Q123. Can we override a variable?

Ans: Nahi. Variables override nahi hote, hide hote hain. Field resolution compile-time par reference type se hoti hai.

Q124. How can we call a parent's overridden method from the subclass?

Ans: 'super.methodName()' se. Lekin grandparent method ko directly 'super.super.method()' se call karna Java mein strictly illegal hai.

Q125. Can an abstract method be overridden?

Ans: Haan, first concrete subclass ko inherited abstract method ko implement/override karna mandatory hota hai.

Q126. Output-based trap: What is the output of the following code?

Ans: obj.show() runtime object B hone ki wajah se 'B' print karega. Lekin obj.only() compile error dega kyunki reference type A ke paas only() method exist nahi karta. Call karne ke liye ((B) obj).only() downcasting zaroori hai.

```
class A { void show() { System.out.println("A"); } }
class B extends A { void show() { System.out.println("B"); } void only() { } }

A obj = new B();
obj.show();     // Prints "B"
// obj.only();  // Compile-time Error!
((B) obj).only(); // Valid after downcasting
```

8. Master Reference & Comparison Tables

Table 8.1: Method Overloading vs Method Overriding

Feature	Method Overloading	Method Overriding
Location	Same class (ya subclass)	Superclass aur Subclass ke beech
Parameters	Must be different	Must be identical
Return Type	Can differ (cannot be sole differentiator)	Same ya Covariant subtype
Polymorphism	Compile-time / Static binding	Runtime / Dynamic binding
Access Modifier	No restrictions	Cannot be more restrictive
Private / Final / Static	Can be overloaded freely	Cannot be overridden
Inheritance	Not mandatory	Strictly mandatory

Table 8.2: this Keyword vs super Keyword

Feature	this Keyword	super Keyword
Reference	Current class instance	Immediate superclass instance
Field Access	this.field accesses current class field	super.field accesses parent hidden field
Method Call	this.method() calls current class method	super.method() calls parent overridden method
Constructor Call	this(...) calls peer constructor in same class	super(...) calls direct parent constructor
Placement Rule	Must strictly be first line in constructor	Must strictly be first line in constructor